

LUMON — Low-Level Design

Module-level implementation design based on the cleaned source tree

Module Map

Module	Key files	Responsibility
Routing	src/router.tsx; src/routes/*	Application routes, auth route, chat route, dashboard and settings
Auth	src/lib/auth.tsx; src/routes/auth.tsx; src/integrations/supabase.ts	Session lifecycle, profile loading, sign-in/sign-up/reset/Auth
Workspace	src/lib/workspace.tsx; src/components/workspace.tsx	Conversation UI, dataset context, sidebar, chat composer and data panel
AI	src/lib/ai.server.ts; src/lib/ai.functions.ts	Gemini planning, profiling, explanation and title generation
Execution	src/lib/pyodide.ts	Browser-side Python execution
Charts	src/components/PlotlyChart.tsx	Interactive Plotly rendering
Export	src/lib/notebook.ts	Jupyter notebook generation
Persistence	Supabase tables	Profiles, conversations, messages, datasets and pinned charts

AI Planning Contract

- planAnalysis validates question, history, datasetContext, role and persona.
- planTurn sends a system instruction plus the latest conversation turns to Gemini.
- Gemini must return JSON with mode, intent, reply and Python.
- Code mode is selected when real calculations, ranking, statistics, trends, charts, cleaning or ML are needed.
- The server never executes generated Python.

Execution Contract

- The browser already owns df. Generated Python must assign its final answer to result.
- Optional chart output is a plain Plotly figure specification.
- The execution layer captures stdout/result and exposes table/chart data.
- If execution fails, the application asks the planner for a corrected script and retries once.
- The explanation step receives the exact execution output and is instructed not to invent or recompute numerical claims.

Persistence Model

- profiles: id, name, email, role, theme, timestamps.
- conversations: id, user_id, title, pinned, archived, timestamps.
- messages: id, conversation_id, user_id, role, content, code_snippet, chart_json, result_json, timestamp.
- datasets: id, user_id, conversation_id, filename, row_count, column_count, schema_metadata, timestamp.
- pinned_charts: id, user_id, conversation_id, title, chart_json, note, timestamp.

Conversation Isolation

- Every persistent record contains a user relationship where appropriate.

- Supabase RLS policies restrict profiles, conversations, messages, datasets and pinned charts to the authenticated owner.
- The active dataframe is maintained by the workspace state keyed to the conversation, so opening a different conversation does not intentionally reuse another conversation's dataframe.

UI Scrolling and Interaction

- ChatWindow uses a dedicated flex child with min-h-0 and overflow-y-auto so long conversations scroll independently.
- Scroll position is tracked to avoid forcibly jumping a user who is reading older messages.
- A latest-message control is available when the user is away from the bottom.
- The application uses responsive sidebars and preserves the composer while the message list scrolls.

Deployment Configuration

- Use .env for Supabase URL/key and server-side GEMINI_API_KEY.
- Never expose GEMINI_API_KEY through VITE_* variables.
- Build with npm run build and serve with the selected Vercel/Node deployment target.
- Git should contain source and .env.example, not production secrets.